IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 16 September 2025, accepted 28 September 2025, date of publication 3 October 2025, date of current version 13 October 2025.

*Digital Object Identifier 10.1109/ACCESS.2025.3617120*

RESEARCH ARTICLE

# A Scalable Framework for Big Data Analytics in Psychological Research: Leveraging Distributed Systems and Cluster Management

**NUR BANU OĞUR[1], CELAL ÇEKEN[2,3], YAVUZ SELİM OĞUR[4], AND ESRA YAZICI[5]**

[1]Department of Software Engineering, Faculty of Computer and Information Sciences, Sakarya University, 54050 Sakarya, Türkiye
[2]Department of Computer Engineering, Faculty of Computer and Information Sciences, Sakarya University, 54050 Sakarya, Türkiye
[3]International Campus, North Kazakhstan University, Petropavlovsk 150000, Kazakhstan
[4]Department of Psychiatry, Serdivan State Hospital, 54050 Sakarya, Türkiye
[5]Department of Psychiatry, Faculty of Medicine, Sakarya University, 54050 Sakarya, Türkiye

Corresponding author: Nur Banu Oğur (nbogur@sakarya.edu.tr)

This work involved human subjects or animals in its research. Approval of all ethical and experimental procedures and protocols was granted by the Ethics Committee of Sakarya University Research Hospital under Approval No. E-43012747-050.04-481672-373.

**ABSTRACT** Anxiety and depression are prevalent psychological disorders that can occur throughout life, with a notably higher prevalence among women during the perinatal period, encompassing pregnancy and the postpartum phase. The early detection and monitoring of these conditions are crucial for timely intervention and improved patient outcomes. Although healthcare analytics has progressed considerably, the extraction of actionable insights from large-scale patient data remains computationally intensive, especially under instant processing constraints. Furthermore, conventional healthcare infrastructures frequently lack the scalability, computational efficiency, and architectural flexibility required to integrate machine learning models into clinical workflows effectively. To address these challenges, the proposed distributed computing framework employs Apache Kafka for instant data streaming, Apache Spark for efficient in-memory machine learning–based analytics, and Kubernetes for orchestrating scalable, fault-tolerant deployment. This architectural configuration facilitates continuous data ingestion, accelerates analytical processing, and ensures system resilience, thereby enabling the timely identification of psychological conditions such as anxiety and depression during the perinatal period. Unlike schematic or batch-only prior work, we provide a production-ready, streaming-first clinical deployment and an empirical scaling analysis linking executors to end-to-end diagnostic latency and resource efficiency. Performance evaluations demonstrate the efficiency and scalability of the proposed system, highlighting its potential for real-world applications in healthcare analytics.

**INDEX TERMS** Real-time data analytics, high-performance computing (HPC), Apache Spark, Kubernetes orchestration, Docker containerization, perinatal psychological disorders.

## I. INTRODUCTION

Big data analytics has garnered significant attention in recent years, driven by the exponential growth of data across various sectors, particularly healthcare [1]. One of the most important concepts in big data analytics is the ability to process large amounts of data quickly and efficiently [2]. The healthcare industry has increasingly relied on big data technologies to improve patient outcomes, optimize resource allocation, and enhance overall efficiency [3]. As the amount of data continues to grow, the ability to process and analyze large datasets in real-time has become essential [4]. This is particularly true for addressing mental health conditions, which have a profound impact on both individual well-being and public health.

In the realm of maternal health, big data analytics is proving to be transformative, particularly in understanding and addressing mental health issues during the perinatal

The associate editor coordinating the review of this manuscript and approving it for publication was Cheolsoo Park.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

period. The perinatal period encompasses pregnancy and the first year after childbirth, a time when women are at an elevated risk for developing mental health disorders such as depression and anxiety [5]. Studies have shown that perinatal mental health conditions are widespread, with the incidence of anxiety disorders ranging from 10% to 23% globally [6], and depression in the perinatal period affecting 10% to 20% of women worldwide [7].

Despite the high prevalence of these conditions, they are often misdiagnosed or undiagnosed, leading to delayed or inadequate treatment [8]. In Turkey, the prevalence of perinatal depression has been estimated to range between 12% and 36%, with variations depending on factors such as the stage of pregnancy and the nature of the woman's work [9], [10].

The consequences of undiagnosed or inadequately treated perinatal mental health conditions are severe and far-reaching. For the mother, untreated depression and anxiety can lead to functional impairments, deterioration of physical health, strained interpersonal relationships, economic hardships, and substance use [11], [12]. These conditions also have profound implications for the infant, as they can lead to developmental issues, health complications, and long-term cognitive and emotional challenges [12], [13]. The need for timely and accurate diagnosis of these conditions is paramount in order to reduce the associated risks and improve outcomes for both mothers and their children.

Perinatal anxiety and depression, which are common and have substantial societal impact, warrant early identification and timely intervention. Big data and AI-supported analytics can help address this need by enabling scalable, real-time risk stratification and decision support [14], complementing rather than replacing comprehensive clinical evaluation by healthcare professionals. Thanks to these systems, the data of any patient using the system can be taken, passed through certain pre-processing processes and instantaneously produced output [15]. With the proposed scalable architecture, the system can accommodate not only data from a single hospital but also patients across the entire province, and even nationwide [16]. By integrating data from multiple sources, big data systems can offer a comprehensive view of mental health trends, identify at-risk populations, and provide evidence-based insights into the effectiveness of different treatment strategies [17]. Given reported prevalence rates and the routine practice of multiple antenatal/postpartum screenings, multi-site programs can accumulate sizeable longitudinal datasets; thus, a scalable, distributed architecture can be pragmatically beneficial even if single-site pilots do not strictly require it. This scalability plays a critical role in addressing the perinatal mental health crisis on a larger scale, ensuring that women in remote areas or with limited access to services receive timely treatment and care, thereby minimizing the potential harms of depression and anxiety in women during the perinatal period.

This paper introduces a scalable big data architecture designed for the diagnostic analysis of psychological disorders—specifically, anxiety and depression—in women during the perinatal period. The system integrates Apache Spark for distributed data processing, MLlib for machine learning–based diagnostic modeling, and Apache Kafka for real-time data streaming and result dissemination. The architecture is deployed within a containerized environment using Docker, with Kubernetes orchestrating resource management and scalability, enabling the system to process large-scale patient data efficiently and reliably.

This work builds upon the findings of a prior study [18], in which hybrid machine learning techniques—comprising feature selection methods and classification algorithms—were employed to identify indicators of perinatal anxiety and depression. Insights from that study informed the design of the present system, particularly in aligning the architectural pipeline with the optimal model configurations. Using the refined dataset from the earlier analysis, a scalable infrastructure was developed to support instant analytics and continuous inference, thereby establishing a generalizable platform for larger-scale psychological data analytics.

The primary objective of this study is to evaluate the computational performance of the proposed architecture, with respect to execution time, memory usage, and CPU efficiency. Special emphasis is placed on analyzing the effect of varying the number of Apache Spark executors on system throughput and responsiveness. These evaluations aim to inform best practices for resource allocation and contribute to the development of high-performance, scalable analytics frameworks for healthcare applications.

The results demonstrate that the proposed platform significantly reduces latency in the diagnostic process, enabling accurate and rapid detection of psychological conditions in pregnant women. These findings highlight the potential for integrating real-time, computer-aided diagnostics into clinical workflows to replace traditionally time-intensive screening procedures. To set expectations, we note that the present cohort is modest in size; our primary contribution is the design and validation of a scalable, production-ready streaming and analytics architecture that can be readily extended to larger, multi-center cohorts and higher-throughput environments.

The remainder of this paper is structured as follows: Section II presents the research goals and a brief overview of the system architecture. Section III describes the system architecture and methodology used in this study. Section IV presents the performance evaluation of the proposed system, including analyses related to execution time, memory usage, and CPU efficiency. Section V concludes the paper by summarizing the key findings and proposing directions for future research.

## II. RESEARCH GOAL

Building on our prior study, which systematically evaluated various machine learning algorithms to determine the most effective approach for diagnosing perinatal anxiety and depression and investigated the applicability of big data

frameworks in this context [18], the present work introduces a containerized and scalable big data architecture that enables real-time analytics and predictive modeling for the identification of psychological disorders, with a specific focus on depression and anxiety during the perinatal period. The proposed architecture uses Apache Spark for data processing, MLlib for machine learning-based diagnosis, and Kafka for data streaming and result delivery. This architecture leverages a distributed system running in Docker containers and orchestrated by Kubernetes to efficiently process large data volumes. It is concluded that with this architecture, instant processing of patient data and timely delivery of accurate results can be ensured. To situate our evaluation, the subsequent section summarizes the system architecture and key implementation details.

### A. SYSTEM ARCHITECTURE AND IMPLEMENTATION

#### 1) APACHE KAFKA

As the number of target systems increases and each system is required to exchange data with multiple others, the complexity of integration escalates substantially. Establishing and maintaining these interconnections introduces several challenges, including data transfer mechanisms, format standardization, and semantic interpretation. Moreover, each additional integration imposes a cumulative processing burden on the source system, potentially degrading its performance. Apache Kafka provides an appropriate solution to these challenges by decoupling producers and consumers and supporting reliable, high-throughput stream ingestion [19].

Apache Kafka (version 2.11) is a message-distribution system capable of handling enormous volumes of data while still providing services to numerous consumers and producers [19]. It has high throughput and low latency. With a minimum delay of a few milliseconds, it can process millions of messages per second. Real-time data streaming is another feature it can enable.

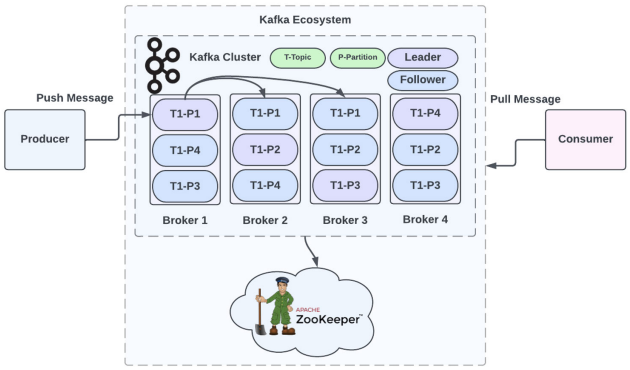



**FIGURE 1.** Core components of the apache kafka architecture.

The main components of Apache Kafka are summarized in Figure 1. In Apache Kafka, all connections take place through topics [20]. Producers are in charge of gathering messages and pushing them to the broker, and the broker is in charge of receiving messages and locally persisting them. Consumers are the true message users; they obtain messages from brokers and process them.

#### 2) APACHE SPARK

For smaller datasets, data analytics tasks can often be performed efficiently without the need for distributed computing or cluster-based architectures. However, as data volume and velocity increase beyond the capacity of a single machine, the use of a distributed computing framework becomes essential. Among such frameworks, Apache Spark has emerged as one of the most widely adopted tools for large-scale data processing, owing to its in-memory computation capabilities, fault tolerance, and scalability. It is a Scala-based open source computing platform [21]. Apache Spark is an extremely quick engine that utilizes resilient distributed datasets, which are in-memory data structures (RDDs). Apache Spark provides versatility as it has diverse libraries, such as SQL, Streaming, Dataframes, Mlib, etc., and can be implemented in different programming languages [22].

##### a: APACHE SPARK STREAMING

Applications for real-time stream processing written in Java, Scala, Python, and R are supported by Apache Spark [22]. Users can manage real-time data streams with Spark Streaming in a scalable, high-throughput, and fault-tolerant way. Discrete streams (DStreams) are created for analysis using Spark Streaming from live data streams. A key data structure of Spark is called a DStream, which is a continuous list of resilient distributed data sets (RDDs) [23]. Each RDD has data from a specific batch interval. The RDDs are then processed by the Spark engine to create the final stream of results in batches, as depicted in Figure 2.

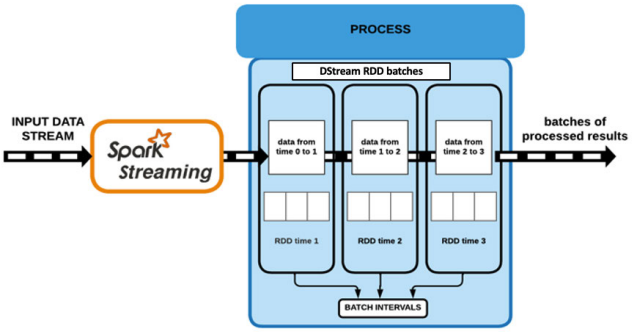



**FIGURE 2.** The general structure of the Apache Spark streaming process.

##### b: APACHE MLLIB

Machine learning based algorithms can be developed using MLlib module of Apache Spark [24]. Popular algorithms like aggregation, regression, and classification are all included in MLlib [25].

While Apache Spark, along with its MLlib and Streaming APIs, provides a comprehensive foundation for large-scale data processing and machine learning, deploying and

managing these components in a distributed setting necessitates a robust orchestration infrastructure [26]. Although Spark can operate in standalone mode, where a single node or a manually configured cluster manages the workload, this approach becomes increasingly inadequate as system complexity grows and the volume of data expands. In large-scale environments, challenges such as dynamic resource allocation, fault recovery, and workload distribution demand a more resilient and automated orchestration solution [27]. Although various orchestration technologies are available, Kubernetes was chosen for its ability to efficiently manage containerized applications, its scalability, and its widespread adoption in cloud-native environments, making it an ideal solution for meeting the demands of distributed computing in big data analytics.

#### 3) KUBERNETES

In the context of this study, which focuses on analyzing psychological disorders in perinatal women using big data analytics, microservice architecture is essential for managing and processing large and complex datasets [28]. As the volume of data grows, the need for a scalable and efficient infrastructure becomes crucial.

Microservices can be thought of as a group of jobs that combine to create the whole application [29]. These microservices have a specific set of activities that they are intended to handle well, and their scope is limited. Inbound page requests are split up into a variety of specialized requests and sent to the appropriate microservices, which then operate as separate processes [30].

The application can be made independent of the host environment by encapsulating each of them in Docker containers in the microservice architecture. Docker supports the bundling of apps into containers for developers [31]. By using Docker containers, applications become highly portable and can run consistently from development to production environments [32]. Containerization plays a critical role in microservices-based architectures, enabling the independent deployment and scaling of individual components across distributed nodes. By using Docker, each microservice can be encapsulated along with its dependencies, facilitating consistent replication and deployment across diverse environments.

However, as the number of containers grows and the system becomes more complex, managing these containers manually becomes increasingly difficult. This is where Kubernetes, an open-source container orchestration platform, comes into play [33]. Kubernetes automates the deployment, scaling, and management of containerized applications, providing a robust solution for orchestrating distributed systems [33]. By managing containers at scale, Kubernetes ensures high availability, efficient resource utilization, and fault tolerance in large, dynamic environments, making it the ideal choice for managing big data analytics platforms. Kubernetes autonomously adjusts the number of containers based on service load, enabling efficient allocation of hardware resources [34]. To accommodate the growing concurrent load, each microservice can be packaged as a Docker image, and the Kubernetes cluster can dynamically scale resources as needed.

In this study, Kubernetes was employed to manage the orchestration of the containerized microservices. Each microservice, responsible for certain tasks, was packaged into a Docker container. One of the containers was dedicated to the Kafka producer, another hosted Apache Spark for MLlib to execute machine learning models, and the third was responsible for the Kafka consumer to handle the results. Kubernetes was then used to automate the deployment, scaling, and management of these containers, ensuring that resources were allocated efficiently as the system scaled.

##### *a: SPARK ON KUBERNETES*

One of the containers in this study was dedicated to Apache Spark, which was responsible for executing machine learning models using Spark's MLlib library. This containerized Spark instance was deployed on Kubernetes to leverage its orchestration capabilities. By utilizing Kubernetes, the Spark container was able to dynamically scale based on the computational demands of the machine learning tasks, ensuring efficient resource utilization.

As shown in Figure 3, when Apache Spark is deployed on Kubernetes, both the driver and executor components are instantiated as pods and managed by the Kubernetes scheduler. Upon submission of a Spark application, a driver pod is created and initialized. The driver then dynamically requests executor pods, establishes communication with them, and orchestrates the execution of the application logic. Upon completion of the application, the executor pods are terminated and removed, while the driver pod transitions to a "Completed" state. Although the driver pod no longer actively participates in computation, its logs remain accessible through the Kubernetes API until it is garbage-collected or manually deleted.

This integration of Spark with Kubernetes successfully enhanced the system's ability to process large datasets and ensured that the machine learning workflows remained robust and scalable throughout the study.

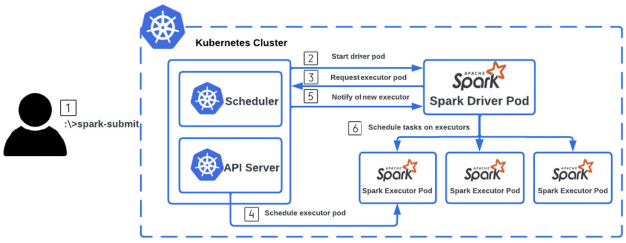



**FIGURE 3.** Lifecycle of a spark application on kubernetes.

## III. METHODOLOGY

This study aims to design and implement a distributed computing framework for big data analytics focusing on

psychological disorders in women during the perinatal period, specifically depression and anxiety. The proposed architecture consists of three main components, each packaged as a Docker container: Kafka Producer, Spark module (Streaming/MLlib), and Kafka Consumer. These components are deployed and managed using Kubernetes, ensuring scalability, fault tolerance, and efficient resource utilization.

### A. DATA DESCRIPTION

Data were obtained from pregnant women who visited the obstetrics outpatient clinic at Sakarya University Training and Research Hospital. Participants completed the Socio-Demographic Data Form (SDDF), the Turkish version of the Perinatal Anxiety Symptom Scale (PASS-TR) [35], and the Edinburgh Postpartum Depression Scale (EPDS) [36]. The SDDF includes questions specifically designed for this study by expert physicians, while PASS-TR and EPDS are validated instruments used to assess anxiety and depression, respectively.

Patients were classified according to their EPDS and PASS scores: EPDS scores below 13 indicated "No Depression" (labeled as '0'), while scores of 13 or higher were classified as depression (labeled as '1'). PASS scores below 17 indicated "No Anxiety" (labeled as '0'), and scores of 17 or higher indicated anxiety (labeled as '1'). Based on these scores, participants were categorized into the groups: "No Anxiety-No Depression," "No Anxiety-Depression," "Anxiety-No Depression," or "Anxiety-Depression." All groups, except "No Anxiety-No Depression," required psychiatric intervention.

For the purpose of this study, participants were grouped into two categories: "No Anxiety-No Depression (0)" and "Anxiety and/or Depression (1)" to assess the applicability of machine learning technologies on big data platforms (Table 1). Individuals in the "Anxiety and/or Depression" category were referred for diagnostic assessment and appropriate management.

**TABLE 1. The choice of options.**

| Anxiety (PASS) | Depression (EPDS) | Label |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 1 | 1 |

The perinatal pregnancy dataset consists of 250 samples, 60 features, and 1 class label. The class label has two possible values: "No Anxiety-No Depression (0)" and "Anxiety and/or Depression (1)." The details of each feature are provided below. The features used as predictive variables include the following:

- Socio-Demographic Data Form (SDDF): age, marital status, physical activity, education, employment, household income, psychological history, and trauma related to the first pregnancy.
- Anxiety levels (PASS), assessing the severity of anxiety symptoms.
- Depression symptoms (EPDS), measuring the severity of depression. All data were utilized for the diagnosis of anxiety and depression. Clinical assessments were conducted by experienced and trained psychiatrists at the study baseline.

### B. DATA PREPROCESSING AND MACHINE LEARNING ANALYSIS

This study is a continuation of the previous study [18]. In this context, various data cleaning and normalization techniques were applied at the preprocessing stage to eliminate missing data and correct abnormal values, and additional transformations were used to enhance feature informativeness. In the machine learning stage, multiple classification algorithms were evaluated and the best-performing Naive Bayes model achieved high accuracy and sensitivity (Table 2).

We previously conducted a systematic comparison across multiple classification paradigms and hyperparameter configurations, with detailed results and settings reported in Table 2 [18]. Based on its favorable accuracy-sensitivity balance and substantially lower computational load for real-time streaming, we selected Naive Bayes for deployment within the present distributed pipeline. In the present work, rather than re-training, we deploy that pre-trained Naive Bayes model for real-time inference within Spark MLlib: records arriving from Kafka are parsed into a structured schema, transformed using the same feature pipeline (e.g., DataFrame encoders and VectorAssembler), scored by the saved PipelineModel via Structured Streaming, and the predictions are published to the "result" topic—clarifying that the focus here is architectural deployment and scalability of the inference pipeline.

The findings indicate that the proposed big data platform can substantially expedite the identification of perinatal anxiety and depression by providing near-real-time, computer-assisted assessments with promising accuracy, supporting and enhancing standard clinical workflows. The present study was designed and refined in light of these observations.

### C. SYSTEM WORKFLOW

This subsection outlines the system workflow, which encompasses the development of a scalable infrastructure designed to produce instant diagnostic results for anxiety and depression in women during the perinatal period. The proposed architecture integrates instant data streaming, distributed processing, and machine learning–based analysis. Performance evaluations were conducted on this infrastructure to assess system efficiency and responsiveness. The overall system workflow, including its core components and data flow, is illustrated in Figure 4. The cluster comprised one master node and two worker nodes implemented as virtual machines, and all IP addresses were replaced with descriptive node names (Master, Worker-1, Worker-2) to avoid disclosing concrete identifiers.

**TABLE 2.** Machine learning model performance metrics.

| Classifier | Accuracy | Sensitivity | Specificity | Precision | AUC |
|---|---|---|---|---|---|
| Decision Tree | 87.20% | 74.05% | 92.22% | 80.05% | 0.822 |
| Naive Bayes | 90.80% | 86.90% | 92.34% | 81.71% | 0.966 |
| K-Nearest Neighbor | 86.80% | 68.57% | 93.89% | 81.95% | 0.900 |
| Random Forest | 88.00% | 69.29% | 95.00% | 86.10% | 0.945 |
| Gradient Boosted Tree | 87.60% | 77.14% | 91.70% | 77.14% | 0.938 |
| Logistic Regression | 84.80% | 73.81% | 88.92% | 74.58% | 0.908 |
| Deep Feed Forward Neural Network | 89.60% | 80.00% | 93.33% | 83.26% | 0.962 |

To enable instant predictions, a streaming-based data processing pipeline was developed using Apache Kafka and Apache Spark. Real patient data obtained from Sakarya Education and Research Hospital was periodically transmitted to a Kafka topic labeled "perinatal". Apache Kafka served as the message delivery backbone, ensuring uninterrupted streaming of data throughout the system.

The architecture comprises three Dockerized components: a Kafka Producer, a Spark module (Streaming/MLlib), and a Kafka Consumer, each deployed within the Kubernetes environment. As depicted in Figure 5, the Kafka Producer ingests raw patient data and sends it to the Kafka Broker. Apache Spark then consumes the incoming data, performs instant processing and analysis using a pre-trained machine learning model via its Streaming and MLlib APIs, and publishes the prediction results to a separate Kafka topic named "result". Finally, the Kafka Consumer retrieves the processed information and presents it through a downstream interface. Kubernetes orchestrates these components, manages their lifecycle, and facilitates scalable, fault-tolerant execution across the cluster.

The Kafka Producer module is designed to ingest real-time data and send it to the system for analysis. In the study, real patient data obtained from Sakarya Training and Research Hospital is streamed to the Kafka topic named "perinatal" at predefined time intervals. The module perpetually transmits patient data, facilitating prompt analysis of psychological health metrics during the perinatal phase, and employs the Kafka messaging system to guarantee dependable and sequential data transmission to the subsequent stage of the pipeline. Additionally, the Producer module is containerized using Docker, allowing seamless deployment within a Kubernetes environment, which ensures scalability, high availability, fault tolerance, and efficient management of large-scale data streams. The following command is used to deploy this pod within the Kubernetes cluster:

```
ubuntu@master:~$ kubectl run --image=nurbanu
    /perinatalproducer perinatalproducer
```

**LISTING 1.** Command for deploying the Kafka Producer module in Kubernetes.

The Apache Spark module serves as the core analytical component of the framework, designed specifically to process data from the Kafka topic named "perinatal" for real-time prediction of whether individuals have depression, anxiety, or both. It consumes real-time patient data from this topic and utilizes the Naive Bayes algorithm, identified in previous studies as the most accurate model, to make these predictions. After making predictions, the module forwards the patient information and the predicted outcomes to another Kafka topic. Fully containerized using Docker, the Spark module integrates seamlessly into a Kubernetes environment, ensuring portability, scalability, and fault-tolerant operation in real-time processing scenarios. The following command is used to initialize this pod within the Kubernetes cluster:

```
helmspark master0:/opt/bitnami/spark$./bin/
    sparksubmit --master k8s://https://<
    cluster-api-endpoint>:6443 --deploy-mode
     cluster --name perinatalspark --class
    Spark --conf spark.executor.instances=5
    --conf spark.kubernetes.authenticate.
    driver.serviceAccountName=helmspark --
    conf spark.kubernetes.container.image=
    nurbanu/spark:sparkcode --conf local:///
    opt/spark/examples/jars/sparkcode.jar
```

**LISTING 2.** Command for deploying the Apache Spark module in Kubernetes.

The Kafka Consumer module serves as the final component in the distributed framework, focusing on real-time data consumption and display. It consumes predictions and associated patient information from the Kafka topic named "result" where the Apache Spark module publishes its outputs. These predictions identify whether individuals have depression, anxiety, or both. The module displays all received data, including predictions and relevant patient details, directly in the console for monitoring and validation purposes. By providing immediate visibility into the predictions and leveraging containerized deployment, the Kafka Consumer module ensures efficient and reliable real-time monitoring. Furthermore, the Kafka Consumer module is fully containerized using Docker, enabling seamless integration into a Kubernetes environment. This containerization ensures scalability, fault tolerance, and efficient management of resources. The following command is used to deploy this pod within the Kubernetes cluster:

```
ubuntu@master:~$ kubectl run --image=nurbanu
    /perinatalconsumer perinatalconsumer
```

**LISTING 3.** Command for deploying the Apache Kafka Consumer module in Kubernetes.

The Spark Standalone Scheduler is suitable for executing small to moderately sized workloads due to its simplicity and minimal configuration overhead. However, for large-scale jobs or production environments requiring elasticity and resource isolation, Kubernetes presents a more robust alternative [33]. Widely adopted for orchestrating containerized applications across distributed hosts, Kubernetes enables dynamic scaling in response to fluctuating workloads, thereby enhancing system responsiveness and resource utilization.

Unlike some cluster managers, the Kubernetes scheduler integrates directly with Spark without requiring the deployment of a separate Spark driver service, resulting in more efficient resource management and reduced overhead. Additionally, Kubernetes supports advanced features such as container orchestration, centralized logging, namespace isolation, and integrated monitoring, making it particularly well-suited for managing distributed data analytics pipelines in complex environments.

In the developed platform, Apache Kafka and Apache Spark—two high-throughput data processing systems—were deployed as standalone containerized services and executed in a distributed configuration on a Kubernetes cluster. As previously described, three additional functional pods, along with the core Kafka cluster components, were incorporated into the system architecture: a Kafka producer to publish real patient data, a Spark-based analytics engine for real-time processing, and a Kafka consumer for delivering the prediction results. Kubernetes orchestrates the execution of these pods by automatically scheduling them across the available worker nodes, thereby optimizing workload distribution. In response to changes in computational demand, the cluster dynamically allocates resources to maintain high availability, minimize latency, and ensure efficient utilization of compute capacity.

## IV. EXPERIMENTAL RESULTS AND DISCUSSIONS

Before presenting the results, we briefly position our measurements with respect to prior Spark performance work: TPC-DS (Transaction Processing Performance Council – Decision Support) studies indicate that Spark on Kubernetes can match YARN performance under proper tuning [37]. Unlike these batch, data-warehouse–style SQL workloads, our experiments target an end-to-end, streaming-first clinical inference pipeline (Kafka⟶ Structured Streaming/MLlib⟶) on Kubernetes, operating on continuously ingested records and emphasizing online latency and executor-scaling under sustained load. Where metrics overlap (e.g., scale-out efficiency), our trends are consistent with the literature; we additionally highlight latency–throughput trade-offs specific to Structured Streaming deployments [38].

To evaluate the performance and scalability of the proposed platform, experiments were conducted in a controlled environment using an HPE ProLiant ML150 Gen.9 Server, which provided the necessary computational resources for running the Kubernetes cluster and supporting the containerized applications. The server's specifications, as outlined in Table 3, include an Intel Xeon E5-2620 v4 CPU with 16 cores, 32 GB of DDR4 RAM, and 1 TB of storage, all running on Linux Ubuntu Server 16.04. These resources were crucial in handling the demands of the experiment, enabling the efficient management of the Kubernetes cluster and execution of tasks across the nodes. When creating a Kubernetes cluster, the cluster is initialized on the master node, which provides a join token for the worker nodes. Using this token, the worker nodes join the cluster. After the connection is established, the master node manages the cluster, while the worker nodes provide resources to run the pods. Once the cluster is created, the connection between all nodes is verified to ensure the system is functioning correctly.

**TABLE 3. Hardware and software specifications of the HPE ProLiant ML150 Gen.9 Server.**

| HPE ProLiant ML150 Gen.9 Server | |
|---|---|
| **CPU** | Intel (R) Xeon CPU E5-2620 v4, 2.1 GHz 16 cores |
| **Memory** | 32 GB DDR4 |
| **Hard disk** | 1 TB |
| **Operating system** | Linux Ubuntu Server 16.04 |

**TABLE 4. Cluster configuration of master and worker nodes.**

| Component | Master Node | Worker Node 1 | Worker Node 2 |
|---|---|---|---|
| **CPU** | 4 cores | 4 cores | 4 cores |
| **Storage** | SATA Port 0, 30 GB | SATA Port 0, 15 GB | SATA Port 0, 15 GB |
| **Memory** | 4 GB | 4 GB | 4 GB |
| **Operating system** | Ubuntu 18.04 | Ubuntu 18.04 | Ubuntu 18.04 |

Table 4 presents the hardware specifications and IP addresses of the master and worker nodes. All nodes were initialized as virtual machines on the same physical server, providing a controlled and resource-aware deployment environment. These virtual machines were configured with identical technical specifications and interconnected to form the Kubernetes cluster.

Various performance tests were conducted to optimize the system within the deployed infrastructure. One such test involved executing Spark workloads on a single Kubernetes node while varying the number of executors to assess scalability. Executors are fundamental processing units in Apache Spark responsible for executing tasks in parallel. Increasing the number of executors enables broader workload distribution, thereby improving concurrency and reducing execution time.

Each executor is allocated a defined share of system resources, including CPU and memory. Accordingly, the total number of executors directly determines the aggregate computational capacity available to the application. Selecting an appropriate number of executors is therefore critical for

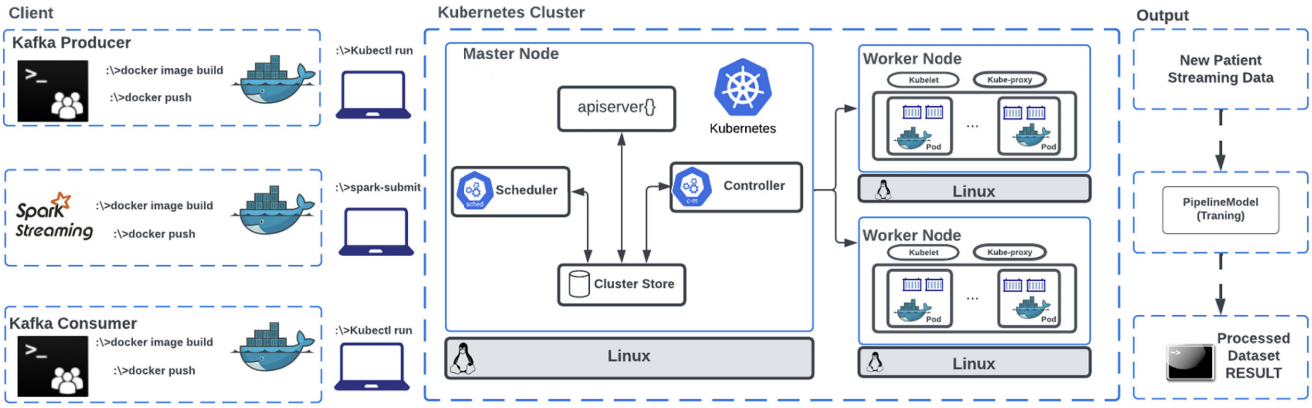



**FIGURE 4.** Workflow and system architecture for the proposed data streaming platform.

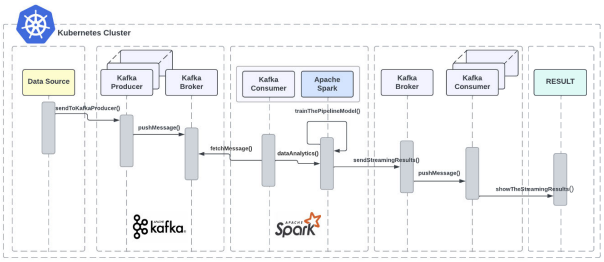



**FIGURE 5.** End-to-End Workflow of the developed data analytics system.

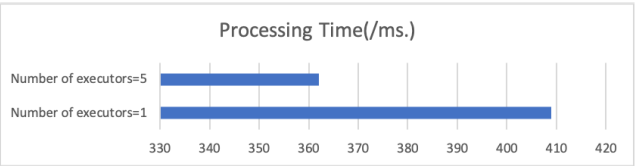



**FIGURE 6.** Processing delay comparison between single-executor and five-executor configurations.

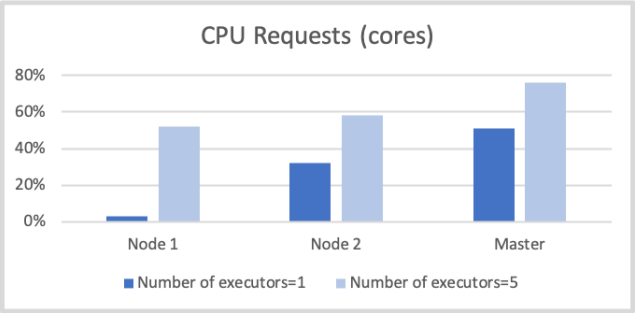



**FIGURE 7.** CPU utilization observed during execution with one and five executors.

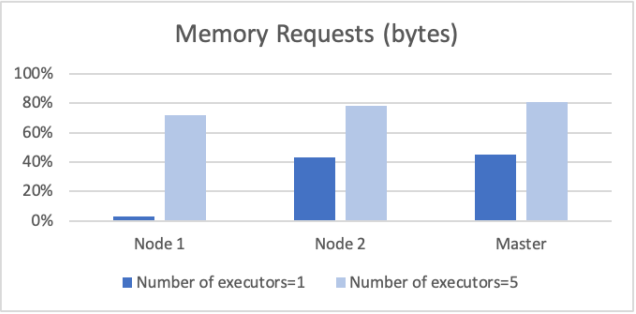



**FIGURE 8.** Memory consumption under different executor configurations.

efficient resource utilization and overall system performance. Proper configuration enhances scalability, allowing the platform to accommodate larger datasets and more complex computational tasks, such as those encountered in big data analytics and machine learning workflows.

To evaluate the trade-offs between performance gains and resource usage, the number of executors was configured to one and five, respectively. The experimental results confirm that increasing the number of executors leads to higher resource consumption, particularly in terms of CPU and memory utilization. As shown in Figure 6, the five-executor configuration yields a noticeable reduction in processing delay compared to the single-executor case. However, this improvement comes at the cost of increased resource usage, as illustrated in Figure 7 for CPU load and Figure 8 for memory consumption. Although the absolute data volume in our experiments is modest, the observed reduction in processing delay with additional executors demonstrates that the pipeline benefits from parallelism, and under larger cohorts and higher message rates these gains are expected to be more pronounced.

While enhanced parallelism reduces execution latency, it must be carefully balanced against resource availability and workload demands. Underprovisioning may lead to degraded performance or task failures, whereas overprovisioning can cause resource exhaustion and inefficiencies in cluster scheduling. Consequently, executor tuning is essential for maintaining stable system operation and achieving optimal throughput under varying computational loads.

## V. CONCLUSION

This study presents a scalable and executable big data analytics framework for detecting anxiety and depression in the perinatal period. By integrating Apache Kafka for real-time data streaming, Apache Spark for streaming-based machine learning analysis, and Kubernetes for scalable deployment, the proposed architecture successfully addresses challenges related to computational efficiency, scalability, and real-time processing in healthcare analytics. Performance testing was conducted on this framework to observe the effects of increasing the number of executors on the results. The performance tests demonstrated that an increase in the number of Spark executors contributed positively to processing speed in parallelization. Additionally, the impact of the increasing number of executors on resource consumption was analyzed, and diagrams illustrating CPU and memory usage were provided. In conclusion, the findings of this study demonstrate that the proposed infrastructure is highly effective and applicable in the healthcare domain.The scalability of the developed big data platform allows for real-time analysis with significant efficiency. More specifically, the results indicate that advanced computer-based systems can support faster and more precise screening for perinatal anxiety and depression, as a complement to established clinical workflows. These findings highlight the potential of big data analytics in improving clinical decision-making and optimizing healthcare workflows. Future research could focus on optimizing computational resource utilization while maintaining scalability and performance. This may involve the development of dynamic resource allocation strategies and adaptive scaling techniques tailored to the demands of real-time healthcare applications. Additionally, ingestion of real-world clinical data and evaluation under sustained and bursty high-throughput conditions-together with controlled fault-recovery assessments are planned as future work to substantiate robustness in production-like environments.

## ACKNOWLEDGMENT

The authors would like to thank the consultant psychiatrists at the Department of Psychiatry for their valuable suggestions and expert validation of the interview questionnaire.

## REFERENCES

[1] L. Yao, Y. Liu, T. Wang, C. Han, Q. Li, Q. Li, X. You, T. Ren, and Y. Wang, "Global trends of big data analytics in health research: A bibliometric study," *Frontiers Med.*, vol. 12, Jul. 2025, Art. no. 1456286.

[2] N. Mehta and A. Pandit, "Concurrence of big data analytics and healthcare: A systematic review," *Int. J. Med. Informat.*, vol. 114, pp. 57–65, Jun. 2018.

[3] R. Al-Dmour, H. Al-Dmour, E. Basheer Amin, and A. Al-Dmour, "Impact of AI and big data analytics on healthcare outcomes: An empirical study in Jordanian healthcare institutions," *Digit. Health*, vol. 11, Jan. 2025, Art. no. 20552076241311051.

[4] J. C. Mayan, M. J. García, A. A. Almerares, M. Househ, and D. R. Luna, "Challenges and potential solutions for big data implementations in developing countries," *Yearbook Med. Informat.*, vol. 23, no. 1, pp. 36–41, 2014.

[5] K. Carlson, S. Mughal, Y. Azhar, and W. Siddiqui, "Postpartum depression," in *StatPearls [Internet]*. Treasure Island, FL, USA: StatPearls Publishing, 2024.

[6] A. R. Mitchell, H. Gordon, J. Atkinson, A. Lindquist, S. P. Walker, A. Middleton, S. Tong, and R. Hastie, "Prevalence of perinatal anxiety and related disorders in low-and middle-income countries: A systematic review and meta-analysis," *JAMA Netw. Open*, vol. 6, no. 11, p. e234, 2023.

[7] C. A. Woody, A. J. Ferrari, D. J. Siskind, H. A. Whiteford, and M. G. Harris, "A systematic review and meta-regression of the prevalence and incidence of perinatal depression," *J. Affect. Disorders*, vol. 219, pp. 86–92, Sep. 2017.

[8] A. K. Sangtiani, M. Mubeen, and A. Irfan, "Insights into perinatal mood and anxiety disorders: Addressing treatment gaps, risk factors, and health outcomes," *WMJ*, vol. 123, no. 6, pp. 497–498, 2024.

[9] P. A. Aslan, N. Aydın, E. Yazıcı, A. N. Aksoy, T. S. Kirkan, and G. A. Daloglu, "Prevalence of depressive disorders and related factors in women in the first trimester of their pregnancies in erzurum, Turkey," *Int. J. Social Psychiatry*, vol. 60, no. 8, pp. 809–817, Dec. 2014.

[10] T. S. Kirkan, N. Aydin, E. Yazici, P. A. Aslan, H. Acemoglu, and A. G. Daloglu, "The depression in women in pregnancy and postpartum period: A follow-up study," *Int. J. Social Psychiatry*, vol. 61, no. 4, pp. 343–349, Jun. 2015.

[11] S. Kendig, J. P. Keats, M. C. Hoffman, L. B. Kay, E. S. Miller, T. A. M. Simas, A. Frieder, B. Hackley, P. Indman, C. Raines, K. Semenuk, K. L. Wisner, and L. A. Lemieux, "Consensus bundle on maternal mental health: Perinatal depression and anxiety," *J. Obstetric, Gynecologic Neonatal Nursing*, vol. 46, no. 2, pp. 272–281, 2017.

[12] J. Slomian, G. Honvo, P. Emonts, J.-Y. Reginster, and O. Bruyère, "Consequences of maternal postpartum depression: A systematic review of maternal and infant outcomes," *Women's Health*, vol. 15, Jan. 2019, Art. no. 1745506519844044.

[13] L. Krause, F. Einsle, J. Petzoldt, H.-U. Wittchen, and J. Martini, "The role of maternal anxiety and depressive disorders prior to and during pregnancy and perinatal psychopathological symptoms for early infant diseases and drug administration," *Early Human Develop.*, vol. 109, pp. 7–14, Jun. 2017.

[14] K. Saqib, A. F. Khan, and Z. A. Butt, "Machine learning methods for predicting postpartum depression: Scoping review," *JMIR Mental Health*, vol. 8, no. 11, Nov. 2021, Art. no. e29838.

[15] K. Batko and A. Ślęzak, "The use of big data analytics in healthcare," *J. Big Data*, vol. 9, no. 1, p. 3, Dec. 2022.

[16] R. Pastorino, C. De Vito, G. Migliara, K. Glocker, I. Binenbaum, W. Ricciardi, and S. Boccia, "Benefits and challenges of big data in healthcare: An overview of the European initiatives," *Eur. J. Public Health*, vol. 29, no. 3, pp. 23–27, Oct. 2019.

[17] F. Ghadirian, "Big data-derived population studies and mental health disparities," *J. Psychosocial Nursing Mental Health Services*, vol. 63, no. 3, pp. 3–4, Mar. 2025.

[18] N. B. Ogur, C. Çeken, Y. S. Ogur, H. U. Yuvaci, A. B. Yazici, and E. Yazici, "Development of an artificial intelligence-supported hybrid data management platform for monitoring depression and anxiety symptoms in the perinatal period: Pilot-scale study," *IEEE Access*, vol. 11, pp. 31456–31466, 2023.

[19] J. Kreps, N. Narkhede, and J. Rao "Kafka: A distributed messaging system for log processing," in *Proc. NetDB*, vol. 11, no. 2011, Athens, Greece, , 2011, pp. 1–7.

[20] C. Martín, P. Langendoerfer, P. S. Zarrin, M. Díaz, and B. Rubio, "Kafka-ML: Connecting the data stream with ML/AI frameworks," *Future Gener. Comput. Syst.*, vol. 126, pp. 15–33, Jan. 2022.

[21] A. Ghodke and G. Kumbhar, "Challenges and opportunities of big data with enhancing cyber security," *Int. J. Sci. Res. Comput. Sci., Eng. Inf. Technol.*, vol. 3, no. 1, pp. 1943–1946, Jan./Feb. 2018.

[22] A. Riesco and J. Rodríguez-Hortalá, "Property-based testing for spark streaming," *Theory Pract. Log. Program.*, vol. 19, no. 4, pp. 574–602, Jul. 2019.

[23] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauly, M. J. Franklin, S. Shenker, and I. Stoica, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proc. 9th USENIX Symp. Networked Syst. Design Implement. (NSDI)*, 2012, pp. 15–28.

[24] D. Siegal, J. Guo, and G. Agrawal, "Smart-MLlib: A high-performance machine-learning library," in *Proc. IEEE Int. Conf. Cluster Comput. (CLUSTER)*, Sep. 2016, pp. 336–345.

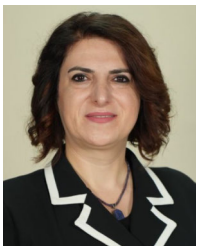
[25] X. Meng, J. K. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. B. Zadeh, M. Zaharia, and A. Talwalkar, "MLlib: Machine learning in apache spark," *J. Mach. Learn. Res.*, vol. 17, no. 34, pp. 1–7, 2015.

[26] R. Furnadzhiev, M. Shopov, and N. Kakanakov, "Efficient orchestration of distributed workloads in multi-region kubernetes cluster," *Computers*, vol. 14, no. 4, p. 114, Mar. 2025.
[27] L. Madison and M. Song, "Cloud-native big data frameworks and their applications," ResearchGate, Aug. 2022. [Online]. Available: https://www.researchgate.net/publication/390236444_Cloud-Native_Big_Data_Frameworks_and_Their_Applications
[28] P. Ataei and D. Staegemann, "Application of microservices patterns to big data systems," *J. Big Data*, vol. 10, no. 1, p. 56, May 2023.
[29] P. Di Francesco, P. Lago, and I. Malavolta, "Architecting with microservices: A systematic mapping study," *J. Syst. Softw.*, vol. 150, pp. 77–97, Apr. 2019.
[30] M. D. Hossain, T. Sultana, S. Akhter, M. I. Hossain, N. T. Thu, L. N. T. Huynh, G.-W. Lee, and E.-N. Huh, "The role of microservice approach in edge computing: Opportunities, challenges, and research directions," *ICT Exp.*, vol. 9, no. 6, pp. 1162–1182, Dec. 2023.
[31] B. S. Kim, S. H. Lee, Y. R. Lee, Y. H. Park, and J. Jeong, "Design and implementation of cloud Docker application architecture based on machine learning in container management for smart manufacturing," *Appl. Sci.*, vol. 12, no. 13, p. 6737, Jul. 2022.
[32] C. Boettiger, "An introduction to Docker for reproducible research," *ACM SIGOPS Operating Syst. Rev.*, vol. 49, no. 1, pp. 71–79, Jan. 2015.
[33] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, omega, and Kubernetes: Lessons learned from three container-management systems over a decade," *Commun. ACM*, vol. 59, no. 5, pp. 50–57, 2016.
[34] S. Ferlito, S. Ippolito, C. Santagata, P. Schiattarella, and G. Di Francia, "A study on an IoT-based SCADA system for photovoltaic utility plants," *Electronics*, vol. 13, no. 11, p. 2065, May 2024.
[35] E. Yazıcı, T. M. Pek, H. U. Yuvacı, E. Köse, S. Cevrioglu, A. B. Yazıcı, A. S. Çilli, A. Erol, and N. Aydın, "Perinatal anxiety screening scale validiy and reliability study in Turkish (PASS-TR validity and reliability)," *Psychiatry Clin. Psychopharmacol.*, vol. 29, no. 4, pp. 609–617, Oct. 2019.
[36] N. Aydin, T. Inandi, A. Yigit, and N. N. S. Hodoglugil, "Validation of the Turkish version of the Edinburgh postnatal depression scale among women within their first postpartum year," *Social Psychiatry Psychiatric Epidemiol.*, vol. 39, no. 6, pp. 483–486, Jun. 2004.
[37] J.-Y. Stephan. (2020). *Apache Spark Performance Benchmarks Show Kubernetes Has Caught Up With Yarn*. Accessed: Sep. 10, 2025. [Online]. Available: https://spot.io/blog/apache-spark-performance-benchmarks-show-kubernetes-has-caught-up-with-yarn/
[38] I. Fedorovych, H. Osukhivska, and N. Lutsyk, "Performance benchmarking of continuous processing and micro-batch modes in Apache Spark structured streaming," in *Proc. CEUR Workshop*, vol. 3896, 2024, pp. 80–90. [Online]. Available: https://ceur-ws.org/Vol-3896/paper5.pdf

**NUR BANU OĞUR** received the B.S. and M.S. degrees in computer engineering and the Ph.D. degree in computer and information engineering from Sakarya University, in 2015, 2018, and 2023, respectively. She is an Academic and Researcher working at the Intersection of Software and Computer Engineering. Since 2023, she has served as an Assistant Professor with the Department of Software Engineering, Sakarya University. Her research interests include real-time data analytics, cloud computing, the Internet of Medical Things (IoMT), and artificial intelligence, through which she contributes to multidisciplinary projects.

**CELAL ÇEKEN** received the Ph.D. degree from Kocaeli University, Kocaeli, Türkiye, in 2003. From 1999 to 2012, he was with Kocaeli University, where he contributed to several projects on wireless networks and scalable web architectures. In 2008, he conducted Post-Doctoral Research with the Department of Electrical Engineering, University of South Florida, USA, for six months. Since September 2012, he has been with the Department of Computer Engineering, Sakarya University, Sakarya, Türkiye. In January 2025, he also began serving as an International Professor at the International Campus of Kozybayev University, Kazakhstan. His current research interests include the Internet of Things (IoT), the IoT data analytics, scalable web architectures, stream processing, and software-defined networking.

**YAVUZ SELİM OĞUR** received the degree from the Faculty of Medicine, Istanbul University, in 2016. He was a Research Assistant with the Psychiatry Department, Sakarya University, from 2018 to 2022. He continued his professional life as the Specialist Doctor with Sakarya Training and Research Hospital after completing the specialization education.



**ESRA YAZICI** received the degree from the Faculty of Medicine, Atatürk University, in 2003. She was a Research Assistant with the Physiology Department, Atatürk University, from 2004 to 2005, and the Psychiatry Department, Atatürk University, from 2005 to 2010. She continued her professional life as the Specialist Doctor with Kocaeli Derince Training and Research Hospital after completing the specialization education. In 2014, she was an Assistant Professor with the Department of Psychiatry, Faculty of Medicine, Sakarya University, and an Associate Professor, in 2015, where she has been a Professor, since 2021.